

Problem Statement: Prefix notation

Procedure:

1. Reverse the given infix expression and swap every '(' with ')' and every ')' with '('.
2. Scan the reversed expression from left to right, one character at a time.
3. If the current character is an operand, append it directly to the output string.
4. If the character is '(', push it onto the stack.
5. If the character is ')', pop and append operators from the stack to the output until '(' is found, then discard the '('.
6. If the character is an operator, pop and append all operators of higher precedence from the stack, then push the current operator.
7. Repeat steps 3-6 until every character has been scanned, then pop and append any operators left on the stack.
8. Reverse the resulting output string to obtain the final prefix expression.

Code:

```
#include <iostream>
#include <stack>
#include <algorithm>
#include <string>
using namespace std;

int getPrecedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return -1;
}

bool isOperand(char c) {
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9');
}

string infixToPrefix(string infix) {
    reverse(infix.begin(), infix.end());

    for (int i = 0; i < infix.length(); i++) {
        if (infix[i] == '(') {
            infix[i] = ')';
        } else if (infix[i] == ')') {
            infix[i] = '(';
        }
    }

    }
```

```

string output = "";
stack<char> st;

for (int i = 0; i < infix.length(); i++) {
    char c = infix[i];

    if (isOperand(c)) {
        output += c;
    } else if (c == '(') {
        st.push(c);
    } else if (c == ')') {
        while (!st.empty() && st.top() != '(') {
            output += st.top();
            st.pop();
        }
        if (!st.empty()) st.pop(); // Pop '('
    } else {

        while (!st.empty() && getPrecedence(c) < getPrecedence(st.top())) {
            output += st.top();
            st.pop();
        }
        st.push(c);
    }
}

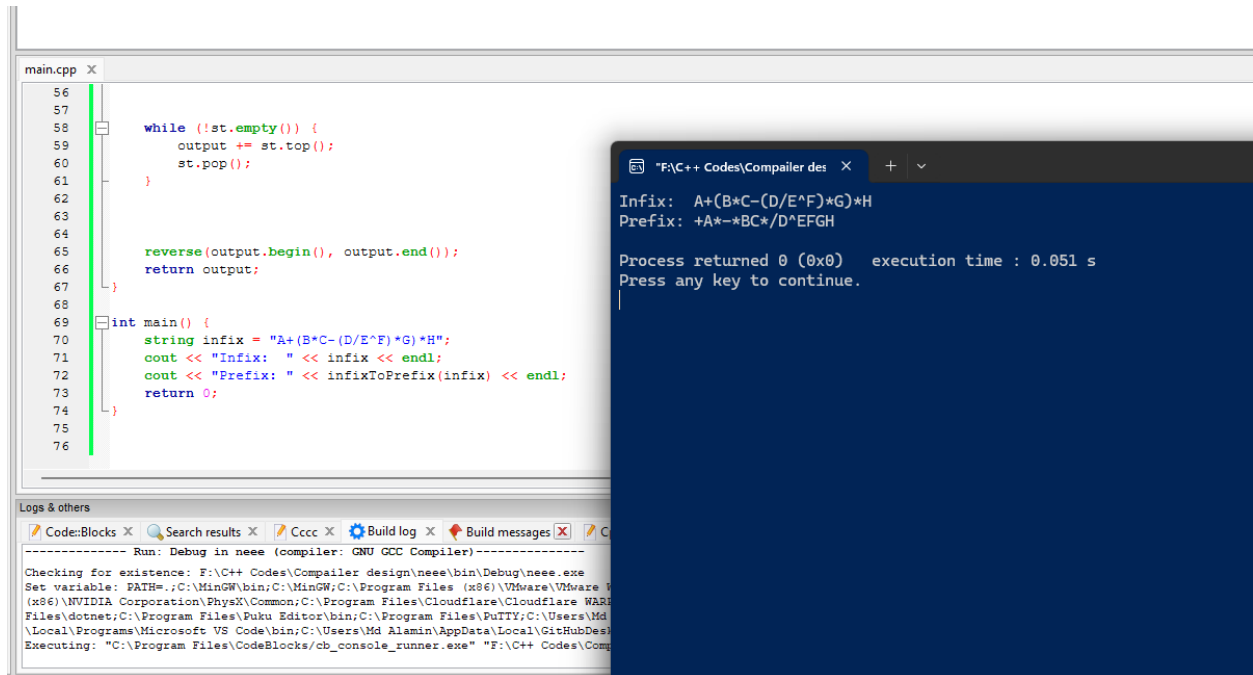
while (!st.empty()) {
    output += st.top();
    st.pop();
}

reverse(output.begin(), output.end());
return output;
}

int main() {
    string infix = "A+(B*C-(D/E^F)*G)*H";
    cout << "Infix: " << infix << endl;
    cout << "Prefix: " << infixToPrefix(infix) << endl;
    return 0;
}

```

Output:



The image shows a C++ IDE with a code editor and a console window. The code in the editor is as follows:

```
56  
57  
58 while (!st.empty()) {  
59     output += st.top();  
60     st.pop();  
61 }  
62  
63  
64  
65 reverse(output.begin(), output.end());  
66 return output;  
67 }  
68  
69 int main() {  
70     string infix = "A+(B*C-(D/E^F)*G)*H";  
71     cout << "Infix: " << infix << endl;  
72     cout << "Prefix: " << infixToPrefix(infix) << endl;  
73     return 0;  
74 }  
75  
76
```

The console window shows the output of the program:

```
"F:\C++ Codes\Compiler des  
Infix: A+(B*C-(D/E^F)*G)*H  
Prefix: +A*-*BC*/D^EFGH  
  
Process returned 0 (0x0)   execution time : 0.051 s  
Press any key to continue.
```

The bottom of the image shows the 'Logs & others' panel with the following text:

```
----- Run: Debug in neee (compiler: GNU GCC Compiler)-----  
Checking for existence: F:\C++ Codes\Compiler design\neee\bin\Debug\neee.exe  
Set variable: PATH=.;C:\MinGW\bin;C:\MinGW\bin\Program Files (x86)\VMware\VMware  
(x86)\NVIDIA Corporation\PhysX\Common;C:\Program Files\Cloudflare\Cloudflare WARE  
Files\dotnet;C:\Program Files\Puku Editor\bin;C:\Program Files\PuTTY;C:\Users\Md  
\Local\Programs\Microsoft VS Code\bin;C:\Users\Md Alamin\AppData\Local\Git\RubDes  
Executing: "C:\Program Files\CodeBlocks\cb_console_runner.exe" "F:\C++ Codes\Comp
```

Problem statement: Prefix notation

Procedure:

1. Scan the given infix expression from left to right, one character at a time.
2. If the current character is an operand (alphanumeric), append it directly to the postfix output string.
3. If the character is '(', push it onto the stack.
4. If the character is ')', pop and append operators from the stack to the output until '(' is found, then discard the '('.
5. If the character is an operator, pop and append operators from the stack whose precedence is greater than or equal to it, then push the current operator.
6. Repeat steps 2-5 until the entire expression has been scanned.
7. After scanning, pop and append any remaining operators from the stack to the output.
8. The final output string is the postfix expression.

Code:

```
#include <iostream>
#include <stack>
#include <string>
#include <cctype>
using namespace std;
int getPrecedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
    return 0;
}

string infixToPostfix(const string& infix) {
    stack<char> s;
    string postfix = "";

    for (char ch : infix) {
        if (isalnum(ch)) {
            postfix += ch;
        }
        else if (ch == '(') {
            s.push(ch);
        }
        else if (ch == ')') {
            while (!s.empty() && s.top() != '(') {
                postfix += s.top();
            }
            s.pop();
        }
    }

    while (!s.empty()) {
        postfix += s.top();
        s.pop();
    }

    return postfix;
}
```

```

        s.pop();
    }
    if (!s.empty()) s.pop(); // Remove '('
}

else {
    while (!s.empty() && getPrecedence(s.top()) >= getPrecedence(ch)) {
        postfix += s.top();
        s.pop();
    }
    s.push(ch);
}
}

while (!s.empty()) {
    postfix += s.top();
    s.pop();
}

return postfix;
}

int main() {
    string infix = "a+b*(c^d-e)^(f+g*h)-i";
    cout << "Infix:   " << infix << "\n";
    cout << "Postfix:  " << infixToPostfix(infix) << "\n";
    return 0;
}

```

Output:

The screenshot displays a C++ IDE with the source code on the left and the execution output on the right. The source code is a C++ program that converts an infix expression to a postfix expression using a stack. The main function uses the expression "a+b*(c^d-e)^(f+g*h)-i". The output window shows the infix and postfix expressions, the execution time, and a prompt to press any key to continue.

```

main.cpp x
38         s.pop();
39     }
40     s.push(ch);
41 }
42 }
43
44 while (!s.empty()) {
45     postfix += s.top();
46     s.pop();
47 }
48
49 return postfix;
50
51
52 int main() {
53     string infix = "a+b*(c^d-e)^(f+g*h)-i";
54     cout << "Infix:   " << infix << "\n";
55     cout << "Postfix:  " << infixToPostfix(infix) << "\n";
56     return 0;
57 }
58

```

```

F:\C++ Codes\Compiler des  x + v
Infix:   a+b*(c^d-e)^(f+g*h)-i
Postfix: abcd^e-fgh*+^*+i-

Process returned 0 (0x0)   execution time : 0.047 s
Press any key to continue.

```

Logs & others

Code:Blocks x Search results x Cccc x Build log x Build messages x CppCh

```

Run: Debug in neee (compiler: GNU GCC Compiler)
-----
Checking for existence: F:\C++ Codes\Compiler design\neee\bin\Debug\neee.exe
Set variable: PATH=.;C:\MinGW\bin;C:\MinGW\bin\Program Files (x86)\VMware\VMware Work
(x86)\NVIDIA Corporation\PhysX\Common;C:\Program Files\Cloudflare\Cloudflare WARP;C:
Files\dotnet;C:\Program Files\Puku Editor\bin;C:\Program Files\PuTTY;C:\Users\Md Ala
\Local\Programs\Microsoft VS Code\bin;C:\Users\Md Alamin\AppData\Local\GitHubDesktop
Executing: "C:\Program Files\CodeBlocks\cb_console_runner.exe" "F:\C++ Codes\Compail

```

Problem Statement: Find the first set of production rules.

Procedure:

1. Read the total number of production rules and store each rule in the form A → alpha.
2. For each non-terminal on the left-hand side, examine the first symbol of its production.
3. If the first symbol is a terminal, add it directly to the FIRST set of that non-terminal.
4. If the first symbol is a non-terminal, recursively compute its FIRST set and add all of its elements to the FIRST set of the original non-terminal.
5. If a production can derive the empty string, include epsilon in the FIRST set of that non-terminal.
6. Repeat the process for every production rule until the FIRST sets of all non-terminals are fully computed.
7. Display the FIRST set of each non-terminal as the final output.

Code:

```
#include <iostream>
#include <cstring>
using namespace std;

char production[10][20];
char firstSet[10][20];
int n;

void findFirst(char ch, int row)
{
    for(int i = 0; i < n; i++)
    {
        if(production[i][0] == ch)
        {
            char symbol = production[i][2];

            // Terminal
            if(!(symbol >= 'A' && symbol <= 'Z'))
            {
                int len = strlen(firstSet[row]);
                firstSet[row][len] = symbol;
                firstSet[row][len + 1] = '\0';
            }
            else
            {
                findFirst(symbol, row);
            }
        }
    }
}
```

```

}

int main()
{
    cout << "Enter number of productions: ";
    cin >> n;

    cout << "Enter productions (e.g. E=TR):\n";

    for(int i = 0; i < n; i++)
        cin >> production[i];

    for(int i = 0; i < n; i++)
    {
        char nonTerminal = production[i][0];
        findFirst(nonTerminal, i);
        cout << "FIRST(" << nonTerminal << ") = { ";

        for(int j = 0; firstSet[i][j] != '\0'; j++)
            cout << firstSet[i][j] << " ";
        cout << "}\n";
    }
    return 0;
}

```

Output:

The screenshot shows a C++ IDE with two main windows. The left window displays the source code for `main.cpp`, which is identical to the code provided in the first block. The right window shows the program's output, which matches the expected results from the code. Below the output window, the 'Logs & others' panel shows the build process, including the execution of the compiler and linker, and the final execution of the program.

```

34      cout << "Enter number of productions: ";
35      cin >> n;
36
37      cout << "Enter productions (e.g. E=TR):\n";
38
39      for(int i = 0; i < n; i++)
40          cin >> production[i];
41
42      for(int i = 0; i < n; i++)
43      {
44          char nonTerminal = production[i][0];
45          findFirst(nonTerminal, i);
46          cout << "FIRST(" << nonTerminal << ") = { ";
47
48          for(int j = 0; firstSet[i][j] != '\0'; j++)
49              cout << firstSet[i][j] << " ";
50          cout << "}\n";
51      }
52      return 0;
53  }
54

```

```

Enter number of productions: 3
Enter productions (e.g. E=TR):
E=TR
R=>TR
T=FY
FIRST(E) = { }
FIRST(R) = { + }
FIRST(T) = { }

Process returned 0 (0x0)   execution time : 71.480 s
Press any key to continue.

```

Logs & others

```

Checking for existence: F:\C++ Codes\Compiler design\final\main.exe
Executing: "C:\Program Files\CodeBlocks\cb_console_runner.exe" "F:\C++ Codes\
Set variable: PATH=C:\MinGW\bin;C:\MinGW\bin;C:\Program Files (x86)\VMware\VMware W
(x86)\NVIDIA Corporation\PhysX\Common;C:\Program Files\Cloudflare\Cloudflare W
Files\dotnet;C:\Program Files\Puku Editor\bin;C:\Program Files\PuTTY;C:\Users\
\Local\Programs\Microsoft VS Code\bin;C:\Users\Md Alamin\AppData\Local\GitHubD

```